

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-13

Выполнил:

Е. Н. Петручик

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

15.05.2026

Брест 2026

Цель работы:

Приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1. Порождающий паттерн(Singleton)

Для последовательности из N целых чисел реализовать обработку:

7) Преподаватель. Класс должен обеспечивать одновременное взаимодействие с несколькими объектами класса Студент. Основные функции преподавателя – ПроверитьЛабораторнуюРаботу, ПровестиКонсультацию, ПринятьЭкзамен, ВыставитьОтметку, ПровестиЛекцию.

Код программы:

```
class Student:
```

```
    def __init__(self, name: str):
        self.name = name
        self.lab_status = {}
        self.exam_grade = None

    def get_name(self) -> str:
        return self.name

    def get_lab_status(self, lab_number: int) -> str:
        return self.lab_status.get(lab_number, "not checked")
```

```
class Professor:
```

```
    _instance = None
    _students = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        if self._students is None:
            self._students = []
```

```

@classmethod
def get_instance(cls):
    if cls._instance is None:
        cls._instance = cls()
    return cls._instance

def check_lab(self, student: Student, lab_number: int):
    if student not in self._students:
        self._students.append(student)
    student.lab_status[lab_number] = "checked"
    print(f"Professor: work {lab_number} of {student.get_name()} checked")

def provide_consultation(self, student: Student, topic: str):
    print(f"Professor: consultation with {student.get_name()} on {topic}")

def take_exam(self, student: Student):
    print(f"Professor: exam taken from {student.get_name()}")

def set_mark(self, student: Student, grade: int):
    student.exam_grade = grade
    print(f"Professor: {student.get_name()} got mark {grade}")

def provide_lecture(self, topic: str):
    print(f"Professor: lecture on {topic}")
    for student in self._students:
        print(f" {student.get_name()} attended")

PROF1 = Professor.get_instance()
PROF2 = Professor.get_instance()

print(f"Same instance? {PROF1 is PROF2}")

STUDENT1 = Student("Ivanov")
STUDENT2 = Student("Petrov")

PROF1.check_lab(STUDENT1, 1)

```

```
PROF1.provide_consultation(STUDENT2, "OOP")
PROF1.take_exam(STUDENT1)
PROF1.set_mark(STUDENT1, 5)
PROF1.provide_lecture("Design Patterns")
```

Рисунки с результатами работы программы

```
Same instance? True
Professor: work 1 of Ivanov checked
Professor: consultation with Petrov on OOP
Professor: exam taken from Ivanov
Professor: Ivanov got mark 5
Professor: lecture on Design Patterns
Ivanov attended
```

Задание 2. Структурный паттерн(Bridge)

7) ДУ автомобиля. Реализовать иерархию автомобилей для конкретных производителей и иерархию средств дистанционного управления. Автомобили должны иметь присущие им атрибуты и функции. ДУ имеет три основные функции – удаленная активация сигнализации, удаленное открытие/закрытие дверей и удаленный запуск двигателя. Эти функции должны отличаться по своим характеристикам для различных устройств ДУ.

Код программы:

```
from abc import ABC, abstractmethod
```

```
class RemoteControl(ABC):
```

```
    @abstractmethod
    def activate_alarm(self):
        pass
```

```
    @abstractmethod
    def toggle_doors(self):
        pass
```

```
    @abstractmethod
    def start_engine(self):
        pass
```

```
class BasicRemote(RemoteControl):
```

```

def __init__(self):
    self.range = 50
    self.feedback = False

def activate_alarm(self):
    print(f"[BasicRemote] Сигнализация активирована (радиус {self.range} м)")

def toggle_doors(self):
    print("[BasicRemote] Двери заблокированы/разблокированы (без
подтверждения)")

def start_engine(self):
    print("[BasicRemote] Двигатель запущен (только если ключ в салоне)")

class AdvancedRemote(RemoteControl):

    def __init__(self):
        self.range = 150
        self.feedback = True

    def activate_alarm(self):
        print(f"[AdvancedRemote] Сигнализация активирована с задержкой 5с (радиус
{self.range} м)")
        if self.feedback:
            print(" -> Звуковой сигнал подтверждения")

    def toggle_doors(self):
        print("[AdvancedRemote] Двери открыты/закрыты (с миганием поворотников)")

    def start_engine(self):
        print("[AdvancedRemote] Двигатель запущен дистанционно (автозапуск)")

class PremiumRemote(RemoteControl):

    def __init__(self):
        self.range = 500

```

```

        self.feedback = True

        self.smartphone_integration = True

    def activate_alarm(self):
        print("[PremiumRemote] Сигнализация активирована с охраной периметра "
              f"(радиус {self.range} м)")
        if self.smartphone_integration:
            print("  -> Уведомление на смартфон: 'Сигнализация включена'")

    def toggle_doors(self):
        print("[PremiumRemote] Двери управляются бесключевым доступом "
              "(с подтверждением на смартфон)")

    def start_engine(self):
        print("[PremiumRemote] Двигатель запущен с климат-контролем (автозапуск по расписанию)")

class Car(ABC):

    def __init__(self, model: str, year: int, remote: RemoteControl):
        self.model = model
        self.year = year
        self.remote = remote
        self.engine_on = False
        self.doors_locked = True
        self.alarm_on = False

    @abstractmethod
    def get_info(self):
        pass

    def remote_activate_alarm(self):
        self.remote.activate_alarm()
        self.alarm_on = True

    def remote_toggle_doors(self):
        self.remote.toggle_doors()
        self.doors_locked = not self.doors_locked

```

```

def remote_start_engine(self):
    self.remote.start_engine()
    self.engine_on = True

def start_manually(self):
    print(f"[{self.model}] Двигатель запущен ключом")
    self.engine_on = True

def lock_doors_manually(self):
    print(f"[{self.model}] Двери заблокированы вручную")
    self.doors_locked = True

class BMW(Car):
    def __init__(self, model: str, year: int, remote: RemoteControl):
        super().__init__(model, year, remote)
        self.brand = "BMW"

    def get_info(self):
        return f"{self.brand} {self.model} ({self.year})"

class Toyota(Car):
    def __init__(self, model: str, year: int, remote: RemoteControl):
        super().__init__(model, year, remote)
        self.brand = "Toyota"

    def get_info(self):
        return f"{self.brand} {self.model} ({self.year})"

class Lada(Car):
    def __init__(self, model: str, year: int, remote: RemoteControl):
        super().__init__(model, year, remote)
        self.brand = "Lada"

    def get_info(self):
        return f"{self.brand} {self.model} ({self.year})"

```

```
if __name__ == "__main__":  
    basic = BasicRemote()  
    advanced = AdvancedRemote()  
    premium = PremiumRemote()  
  
    bmw_x5 = BMW("X5", 2023, premium)  
    toyota_camry = Toyota("Camry", 2022, advanced)  
    lada_vesta = Lada("Vesta", 2021, basic)  
  
    print("Тестирование BMW X5 с PremiumRemote")  
    print(bmw_x5.get_info())  
    bmw_x5.remote_activate_alarm()  
    bmw_x5.remote_toggle_doors()  
    bmw_x5.remote_start_engine()  
    print()  
  
    print("Тестирование Toyota Camry с AdvancedRemote")  
    print(toyota_camry.get_info())  
    toyota_camry.remote_activate_alarm()  
    toyota_camry.remote_toggle_doors()  
    toyota_camry.remote_start_engine()  
    print()  
  
    print("Тестирование Lada Vesta с BasicRemote")  
    print(lada_vesta.get_info())  
    lada_vesta.remote_activate_alarm()  
    lada_vesta.remote_toggle_doors()  
    lada_vesta.remote_start_engine()  
    print()  
  
    print("Меняем пульт у Lada на PremiumRemote")  
    lada_vesta.remote = premium  
    lada_vesta.remote_activate_alarm()  
    lada_vesta.remote_start_engine()
```


Рисунки с результатами работы программы

```
Тестирование BMW X5 с PremiumRemote
BMW X5 (2023)
[PremiumRemote] Сигнализация активирована с охраной периметра (радиус 500 м)
  -> Уведомление на смартфон: 'Сигнализация включена'
[PremiumRemote] Двери управляются бесключевым доступом (с подтверждением на смартфон)
[PremiumRemote] Двигатель запущен с климат-контролем (автозапуск по расписанию)

Тестирование Toyota Camry с AdvancedRemote
Toyota Camry (2022)
[AdvancedRemote] Сигнализация активирована с задержкой 5с (радиус 150 м)
  -> Звуковой сигнал подтверждения
[AdvancedRemote] Двери открыты/закрыты (с миганием поворотников)
[AdvancedRemote] Двигатель запущен дистанционно (автозапуск)

Тестирование Lada Vesta с BasicRemote
Lada Vesta (2021)
[BasicRemote] Сигнализация активирована (радиус 50 м)
[BasicRemote] Двери заблокированы/разблокированы (без подтверждения)
[BasicRemote] Двигатель запущен (только если ключ в салоне)

Меняем пульт у Lada на PremiumRemote
[PremiumRemote] Сигнализация активирована с охраной периметра (радиус 500 м)
  -> Уведомление на смартфон: 'Сигнализация включена'
[PremiumRemote] Двигатель запущен с климат-контролем (автозапуск по расписанию)
```

Задание 3. Поведенческий паттерн(Command)

7) Проект «Пиццерия». Реализовать формирование заказ(а)ов, их отмену, а также повторный заказ с теми же самыми позициями.

Код программы:

```
from abc import ABC, abstractmethod
from typing import List, Optional

class Pizza:

    def __init__(self, name: str, price: float):
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} ({self.price} rub.)"

    def get_price(self) -> float:
```

```
    return self.price
```

```
class Order:
```

```
    def __init__(self, order_id: int):
```

```
        self.id = order_id
```

```
        self.pizzas: List[Pizza] = []
```

```
        self.total = 0.0
```

```
        self.status = "created"
```

```
    def add_pizza(self, pizza: Pizza):
```

```
        self.pizzas.append(pizza)
```

```
        self.total += pizza.price
```

```
    def confirm(self):
```

```
        self.status = "confirmed"
```

```
        print(f"Order #{self.id} confirmed. Total: {self.total} rub.")
```

```
    def cancel(self):
```

```
        self.status = "cancelled"
```

```
        print(f"Order #{self.id} cancelled.")
```

```
class Command(ABC):
```

```
    @abstractmethod
```

```
    def execute(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def undo(self):
```

```
        pass
```

```
class CreateOrderCommand(Command):
```

```
    def __init__(self, pizzeria: 'Pizzeria', pizzas: List[Pizza]):
```

```

        self._pizzeria = pizzeria
        self._pizzas = pizzas
        self._order: Optional[Order] = None

    def execute(self):
        self._order = self._pizzeria.create_order(self._pizzas)
        return self._order

    def undo(self):
        if self._order:
            self._pizzeria.cancel_order(self._order.id)

class CancelOrderCommand(Command):

    def __init__(self, pizzeria: 'Pizzeria', order_id: int):
        self._pizzeria = pizzeria
        self._order_id = order_id
        self._cancelled_order: Optional[Order] = None

    def execute(self):
        self._cancelled_order = self._pizzeria.cancel_order(self._order_id)
        return self._cancelled_order

    def undo(self):
        if self._cancelled_order:
            self._pizzeria.restore_order(self._cancelled_order)

class RepeatOrderCommand(Command):

    def __init__(self, pizzeria: 'Pizzeria', original_order: Order):
        self._pizzeria = pizzeria
        self._original_order = original_order
        self._new_order: Optional[Order] = None

    def execute(self):
        if self._original_order.status == "cancelled":

```

```

        print(f"Cannot repeat cancelled order #{self._original_order.id}")
        return None

    self._new_order =
self._pizzeria.create_order(self._original_order.pizzas.copy())

    print(f"Repeated order #{self._original_order.id} -> new order
#{self._new_order.id}")
    return self._new_order

def undo(self):
    if self._new_order:
        self._pizzeria.cancel_order(self._new_order.id)

class Pizzeria:

    def __init__(self, name: str):
        self.name = name
        self.orders: List[Order] = []
        self._next_id = 1
        self._history: List[Command] = []

    def create_order(self, pizzas: List[Pizza]) -> Order:
        order = Order(self._next_id)
        self._next_id += 1
        for pizza in pizzas:
            order.add_pizza(pizza)
        order.confirm()
        self.orders.append(order)
        return order

    def cancel_order(self, order_id: int) -> Optional[Order]:
        for order in self.orders:
            if order.id == order_id and order.status != "cancelled":
                order.cancel()
                return order
        print(f"Order #{order_id} not found or already cancelled.")
        return None

```

```

def restore_order(self, order: Order):
    if order.status == "cancelled":
        order.status = "confirmed"
        print(f"Order #{order.id} restored.")
        if order not in self.orders:
            self.orders.append(order)

def execute_command(self, command: Command):
    result = command.execute()
    self._history.append(command)
    return result

def undo_last_command(self):
    if self._history:
        cmd = self._history.pop()
        cmd.undo()
        print("Last operation undone.")
    else:
        print("No commands to undo.")

def show_orders(self):
    print(f"\n=== Orders in {self.name} ===")
    if not self.orders:
        print("No orders.")
    for order in self.orders:
        pizzas_str = ", ".join(str(p) for p in order.pizzas)
        print(f"  #{order.id}: [{pizzas_str}] - {order.total} rub.
({order.status})")

if __name__ == "__main__":
    pizzeria_obj = Pizzeria("Pizza Hut")

    margherita = Pizza("Margherita", 450)
    pepperoni = Pizza("Pepperoni", 550)
    four_cheese = Pizza("Four Cheese", 600)

```

```
print("\nCreating order")
create_cmd = CreateOrderCommand(pizzeria_obj, [margherita, pepperoni])
order1 = pizzeria_obj.execute_command(create_cmd)

print("\nCancelling order")
cancel_cmd = CancelOrderCommand(pizzeria_obj, order1.id)
pizzeria_obj.execute_command(cancel_cmd)

print("\nCreating another order")
create_cmd2 = CreateOrderCommand(pizzeria_obj, [four_cheese])
order2 = pizzeria_obj.execute_command(create_cmd2)

print("\nRepeat order #2")
repeat_cmd = RepeatOrderCommand(pizzeria_obj, order2)
order3 = pizzeria_obj.execute_command(repeat_cmd)

print("\nUndo last command (repeat)")
pizzeria_obj.undo_last_command()

pizzeria_obj.show_orders()

print("\nUndo cancellation (restore order #1)")
pizzeria_obj.undo_last_command()
pizzeria_obj.show_orders()
```

Рисунки с результатами работы программы

```
Creating order
Order #1 confirmed. Total: 1000.0 rub.

Cancelling order
Order #1 cancelled.

Creating another order
Order #2 confirmed. Total: 600.0 rub.

Repeat order #2
Order #3 confirmed. Total: 600.0 rub.
Repeated order #2 -> new order #3

Undo last command (repeat)
Order #3 cancelled.
Last operation undone.

=== Orders in Pizza Hut ===
#1: [Margherita (450 rub.), Pepperoni (550 rub.)) - 1000.0 rub. (cancelled)
#2: [Four Cheese (600 rub.)) - 600.0 rub. (confirmed)
#3: [Four Cheese (600 rub.)) - 600.0 rub. (cancelled)

Undo cancellation (restore order #1)
Order #2 cancelled.
Last operation undone.

=== Orders in Pizza Hut ===
#1: [Margherita (450 rub.), Pepperoni (550 rub.)) - 1000.0 rub. (cancelled)
#2: [Four Cheese (600 rub.)) - 600.0 rub. (cancelled)
#3: [Four Cheese (600 rub.)) - 600.0 rub. (cancelled)
```

Вывод:

Я приобрёл навыки применения паттернов проектирования при решении практических задач с использованием языка Python.